

Assignment 2

Parallel Computing and Data Processing

Luca Stradiotti
luca.stradiotti@kuleuven.be

Projects are independent: no working together! You must come up with how to solve the problem independently. Do not discuss specifics of how you structure your solution, etc. You cannot share solution ideas, pseudocode, code, reports, etc.

Beyond what is provided, you may not use any other code that is available online. The use of language models such as ChatGPT or any other AI-based systems for completing this programming assignment is strictly prohibited. If you are unsure about the policy, ask the professor in charge or the TAs.

1 Introduction

The popular version of Moore’s law is that computers double in efficiency roughly every two years, leading to exponentially more computing power over time. This was true for a very long time. However, sometime in the last decade, computer cores have stopped getting faster. The answer to the “end of Moore’s Law” is Parallel and Distributed Computing. The goal of this project is to gain experience working with different ways to parallelize and distribute computation. Specifically, we will look at two techniques: parallelism within a single core (i.e., SIMD) and parallelism with multiple independent processes (i.e., distributed computing).

2 SIMD (6 Points)

You will write a simple SIMD program to compute the prediction for an entire test set for a linear regression model. Concretely, you will compare two implementations:

1. A standard single-threaded scalar version of the program.
2. A single-threaded SIMD version of the function. For this function, you must use the CPU intrinsic functions in your implementation (see exercise session 2). That is, you cannot rely on the compiler optimizations to perform automatic vectorization.

Implement these two variants in the C++ functions `evaluate_scalar` and `evaluate_simd` respectively. The data is available from the departmental computers at `/cw/bdap/assignment2/simd`.

You must use the following commands to compile and run your code:

```
% g++ -O1 -g -mavx -mavx2 -mfma lin_regression.cpp -o test && ./test
```

For the efficiency of your code, keep in mind two things. First, the more vectorization you can achieve the better. Second, think about what operations cost the most time in SIMD and try to minimize those. You are not allowed to use multi-threading, because we want the coding to take less time and get you to focus on the SIMD part.

3 Distributed Computing (12 Points)

There are many different models for distributed computing. Initially, the MapReduce framework as implemented in, e.g., Hadoop, was extremely popular. Current data-handling frameworks, such as Spark, build upon the MapReduce framework as their core distributed programming model. In the second part of this assignment, we will put Spark to good use for analyzing basketball data. You will perform this analysis on a Hadoop cluster. Basic instructions on how to run code on this cluster are available on the Toledo assignment page. Further details will be provided during exercise session 5: it is essential that you attend and complete this exercise session.

3.1 Dataset

You will be working on SportVU movement tracking data¹ for 84 games played by the Golden State Warriors and the Cleveland Cavaliers between October 27, 2015 (the start of the 2015/16 NBA season) and January 23, 2016.² A sample of a single game is available from the departmental computers at `/cw/bdap/assignment2/nba_movement_data`. The full dataset is available on the distributed file system (DFS) at `/data/nba_movement_data`. For our analysis, we use the following files for each game:

Play-by-play (pbp) data (`./events/<game_id>.csv`) This dataset comes in a CSV file with a unique row for each “event” in the game. These events correspond to typical basketball actions such as a dribble, pass, shot, foul, rebound, etc. Each event element has attributes describing the type of event, the time of the event, and the player who completed the action.

Trajectory data (`./moments/<game_id>.csv`) This data provides precise coordinates for all ten players and the ball at a rate of 25 times per second. All the coordinates are expressed in feet. We will refer to these spatial snapshots as “moments”. Each row of the dataset describes the location of a single player or the ball in a moment and has the following format:

```
<team_id> <player_id> <x_loc> <y_loc> <radius> <game_clock> <shot_clock> <quarter> <game_id>
<event_id>
```

The ball’s team and player ID is `-1` and has a value for the z coordinate (`radius`), while players are only described by x and y coordinates. The positional data comes along with other information like the time remaining on the shot and game clock (in seconds), as well game, team and event ids. These ids match up with the numbers from the play-by-play data and can thus be used to link moments to events.

We also provide an overview of all games (`./games.csv`), players (`./players.csv`) and teams (`./teams.csv`) that can be used to infer the name that matches with each ID. Finally, the (`./minutes_played.csv`) file provides the number of seconds that each player was on the court in each game.

A key challenge when working with big data is understanding its structure so that the ensuing analyses are

¹If you want to learn more about how teams are using this data, here’s an article on how the Raptors use it to analyze their games: <http://grantland.com/features/the-toronto-raptors-sportvu-cameras-nba-analytical-revolution/>.

²Five games are missing: Rockets vs. Warriors on 30/10/2015, Cavaliers vs Nets on 28/11/2015, Spurs vs Cavaliers on 14/01/2016, Pistons vs Warriors on 16/01/2016, Cavaliers vs Clippers on 21/01/2016, and Bulls vs Cavaliers on 24/01/2016

correct.

3.2 Spark

In this part of the assignment, you will write multiple Spark programs in Java to answer some queries about the data. You are required to use SparkSQL (<https://spark.apache.org/docs/latest/sql-programming-guide.html>) for this part of the assignment; further details on how to use SparkSQL are provided during the lectures.

3.2.1 Distance Travelled

For this part of the assignment, you will write a Spark program to compute the distance traveled by each player. You need to compute the Euclidean distance in the xy-plane according to the formula given below:

$$TotalDistance = \sum_{i=0}^{M-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2}, \quad (1)$$

where you first calculate the distance between each pair of two consecutive moments and aggregate them in order to obtain the total distance traveled by that player. You do not need to handle cases where a player appears to 'jump' from one location to another between two consecutive timestamps. Finally, use again the `./minutes_played.csv` file to normalize the obtained distances for the time that a player has been on the pitch and express the final result as the average distance that a player covers per quarter (i.e., 12 minutes):

$$TotalDistance/quarter = \frac{TotalDistance * 12}{minutes\ played}, \quad (2)$$

Implement your solution and test it locally on the single-game sample dataset. Once you have developed and tested your implementation on the sample dataset, you should run it on the cluster on the complete dataset.

The program must output a single **space-separated** `distance_per_player.csv` file with the following columns:

`<player_ID> <average_distance_traveled_per_quarter>`

where `<average_distance_traveled_per_quarter>` is expressed in meters, rounded to the nearest integer, and averaged over the season for each player.

3.2.2 Ball Possession

For this part of the assignment, you will write a Spark program to compute the amount of time a player is in possession of the ball. Determining ball possession from tracking data can be challenging, since multiple players may be close to the ball at the same time. To simplify this task, a player is considered to be in possession of the ball if (i) the player is within a radius of 0.5 meters from the ball, and (ii) no other player is within this radius.

A ball possession phase for a player starts when these conditions are met and ends as soon as either condition is no longer satisfied. Your program should process the tracking data according to this rule and compute for each player:

- the total distance traveled while in possession of the ball;
- the total time in possession of the ball;
- the total time the player was on the court;
- the percentage of time the player was in possession of the ball with respect to the total amount of time he was on the court, rounded to two decimal places;

The program must output a single **space-separated** `possession_per_player.csv` file with one line per player, containing the following columns:

```
<player_ID> <distance_traveled_with_ball_possession> <time_ball_possession>
<total_time_on_court> <percentage_ball_possession>
```

3.2.3 Clutch time efficiency

In the NBA, **clutch time** refers to the final five minutes of a close game (including overtime) when the score difference is five points or fewer. These high-pressure moments are often used to extract key performance statistics, as some players consistently elevate their performance under pressure. In this part of the assignment, you will measure players' clutch performance by analyzing shots attempted and scored. In the NBA, there are two types of field shots based on the distance from which the shot is taken:

- 2-point field shots, when a player shoots from inside the three-point line;
- 3-point field goals, attempted from beyond this three-point line.

To simplify the computation, you can assume that the three-point line is a perfect semi-circle with radius 6.71 meters centered at the basket.

For each player, you need to compute two things.

1. First, compute the total number of 2-point and 3-point field goals attempted by each player across all included matches during clutch time. Since the play-by-play (pbp) data can be inaccurate, you must determine whether a shot was a 2-point or a 3-point attempt by computing the distance from the basket. This requires combining the play-by-play (pbp) data with the tracking data: use the pbp data to identify when a shot is attempted and the tracking data to determine the location from which the shot was actually taken. Note that you should calculate the distance of the player from the basket at the moment the shot is taken, not when the make or miss is registered. Also be aware that the timestamps in the pbp and tracking data may not be perfectly synchronized, so you will need to implement a method to accurately align these and find the shot location.
2. Second, compute each player's clutch efficiency, i.e., the fraction of shots that resulted in a basket (i.e., a make) during clutch time considering separately 2-point and 3-point shots. To do this, you need to divide for each shot category the number of made field goals by the total number of attempted shots during clutch time, e.g., for 2 point shots this is:

$$\text{2point_efficiency} = \frac{\text{Total Clutch 2Point Goals}}{\text{Total Clutch 2Point Shots}}. \quad (3)$$

Your program must output a single **space-separated** file named `clutch_efficiency.csv` with the following columns:

`<player_ID> <2pts_efficiency> <nb_2pts_shots> <3pts_efficiency> <nb_3pts_shots>`

The file has to be sorted by decreasing `2pts_efficiency`, i.e., the player with the highest 2-point efficiency should rank first, and the `2pts_efficiency` and the `3pts_efficiency` must be rounded to two decimal places.

4 Report (2 Points)

Part of this course's goal is to gain insights into the interplay between large amounts of data and algorithms. You must also include a small report. The report must be submitted in PDF, for which you **must** use the provided LaTeX template on Toledo. **Please remember to include your name and student number on your report.** All that is required in this report is:

1. Report the average run time and its standard deviation, as well as the speedup of the SIMD implementation compared to the scalar implementation for each dataset. To obtain reliable estimates, repeat the experiments multiple times. You need to present the results as a table or a graph.
2. Include a histogram showing the distribution of the fraction of time each player held possession of the ball. The x-axis should represent the fraction of possession time (normalized by total time on court), and the y-axis should represent the number of players.
3. Include a shot chart that visualizes all field-goal attempts taken by Stephen Curry during clutch time. You should display every shot attempt, marking scored and missed shots using distinct symbols or colors so that they can be easily distinguished on the diagram (the legend should specify which marker refers to the shots missed and which to the one scored). As players shoot toward both baskets during a game, you must transform all shot locations so that they are shown as if they were taken toward the basket on the right side of the final figure. This requires flipping the coordinates across the middle of the court for shots originally taken toward the left basket. Figure 1 provides you an example chart containing one missed and one scored shot. You can use `mplbasketball` to draw the basketball court.

5 (Very) Important remarks

- Please look at the FAQ on Toledo before mailing with questions.
- You are graded on code (1) correctness, (2) efficiency, and (3) readability and documentation. Note that when evaluating correctness, we can test your code on other datasets for, e.g., edge cases.
- The NBA tracking data **cannot** be redistributed under any circumstances.
- Do **NOT** run anything on the clusters before making sure it runs locally (at least on a smaller dataset). The clusters are shared resources.
- **Kill your jobs** on the cluster, if you think they are going to crash or run for too long. For that use `hadoop job -list` and `hadoop job -kill <job id>`.
- Scaling up to the large dataset is not trivial and may require some work. Make sure you have a decent implementation for the smaller dataset single-game dataset before working on the scalability on the full dataset.

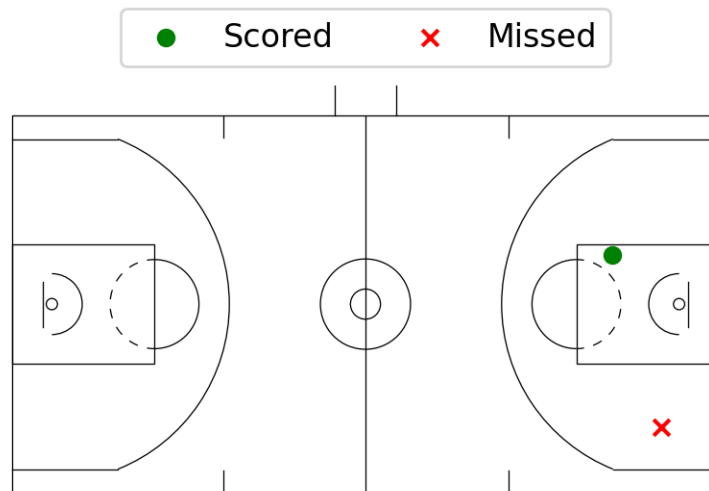


Figure 1: The green dot represents a scored shot, while the red cross a missed one (which happened on the far half of the court).

6 Turning in your code and report

Deadline The assignment should be handed in on Toledo before **Friday, March 27 at 17:00**. Late submissions will not be accepted.

Deliverables You must upload an archive (.zip or .tar.) containing the following files:

```
assignment2
├── report.pdf
├── SIMD
├── lin_regression.cpp
├── Spark
│   ├── distance_per_player.sh
│   ├── distance_per_player.csv
│   ├── possession_per_player.sh
│   ├── possession_per_player.csv
│   ├── clutch_efficiency.sh
│   ├── clutch_efficiency.csv
│   └── src/
│       └── ...
```

- the report (as pdf)
- the `lin_regression.cpp` file for part one
- for each task of part two:

- a runnable bash script to compile your source code (i.e., the one submitted in the src folder) and execute it (on the cluster).
- a csv file containing the requested output
- your source code for the Spark part in a sub-directory called **src** (which may contain more subdirectories if necessary). **Submitting your source code is essential for evaluating the correctness of your solution. Failure to submit the source code will result in an automatic score of zero for the Spark part.**

Evaluation Your solution will be evaluated based on the correctness of your results, the runtime efficiency, and the style of your code.

Automated grading Automated tests are used for grading. Therefore you must follow these instructions:

- Use the exact filenames as specified in the assignment.
- Do not change the provided interface/skeletons, this includes the constructor.
- Your code must run on the departmental machines, this can be done over ssh.

If you fail to follow these instructions, you will lose points.

Good luck!